

Name: ycchiu226

Date: September 30, 2026

Topic: Object-Oriented Programming (OOP)

1. Today's Challenge

Core concept from today's OCW lecture:

Classes, instance attributes, data abstraction, operator overloading, method reuse, and exception handling.

AI-generated coding challenge title:

Design a Playlist Class — Managing Unique Songs with OOP

2. My Initial Approach — Before AI Help

My approach:

I planned to use a list as the internal representation for storing songs. Before every insertion, I would check whether the playlist already contained the song to prevent duplicates.

For combining playlists, I would add each song from the second playlist into the first playlist's contents while checking for duplicates. For removal, I would first check whether the song existed and raise a `ValueError` if it did not.

3. AI Tutor Help

Did you ask the AI Tutor for help?

☐ No — I solved it independently

☒ Yes — I received one or more hints

The most useful hint/question from AI was:

When implementing `__add__()`, consider which playlist should receive the new songs. Directly modifying either original playlist might produce the correct combined result while violating the requirement that the original playlists remain unchanged.

It helped me realize that:

The addition operation should create a new `Playlist` instance and populate it using the existing `add()` method. This preserves the original playlists and reuses the uniqueness-checking logic.

4. My Revision

Did you change your approach or code after interacting with AI?

☒ No

☐ Yes

What did you change, and why?

I did not need to change my approach or code because I had already planned to create a new playlist during the addition process. The AI's hint confirmed that this design correctly preserves the original playlists.

5. Verification

My final program:

☒ Passed the provided examples

☒ Passed additional edge cases

☐ Still has unresolved problems

One edge case I tested:

Input:

```
p1 = Playlist()
p1.add("A")
p1.add("B")

p2 = Playlist()
p2.add("B")
p2.add("C")

p3 = p1 + p2

print(p3)
print(p1)
print(p2)
```

Expected output:

```
A, B, C
A, B
B, C
```

Actual output:

```
A, B, C
A, B
B, C
```

6. One-Minute Reflection

What idea from the OCW lecture did you transfer to this new problem?

I transferred the `intSet` class's approach of maintaining a collection through methods that enforce its constraints, along with the `Fraction.__add__()` pattern for operator overloading.

I also applied instance attributes, data abstraction, method reuse, and exception handling to design the `Playlist` class.

One thing I understand better now:

I understand how OOP allows a class to encapsulate data and define meaningful operations through its interface. In particular, I learned how to implement `__add__()` to combine two objects while returning a new instance without modifying the original objects.

I also understand why repeatedly checking list membership during insertion results in $O((n+m)^2)$ worst-case time complexity when combining two playlists.

One thing I am still unsure about:

How Python implements the dot operator `.` internally, particularly how attribute lookup and method binding work. I am also curious about how Python's object references differ from C++ pointers and why Python does not require a separate `->` operator.